CSCI 7000 · Final Presentation

ReactRefactor

An Agentic Pipeline for Automated React Code Smell Remediation using LangGraph, Tactic-Constrained Planning, and Multi-Gate Verification

Jayanth Sibbala · Ved Mulik · Soham Sant

CSCI 7000-05 SPRING

→

THE PROBLEM

Why is Automated React Refactoring Hard?

1 Silent Behavioral Regressions

Dropping a React context provider passes TypeScript but silently breaks form validation at runtime — invisible to static analysis.

2 State & Side-Effect Complexity

Hooks, `useEffect` dependencies, memoization — moving code between components can introduce stale closures and referential equality bugs.

3 Component Interdependence

Props, shared context, and rendering hierarchies create tight coupling. Changing a prop surface must be consistent with every call site.

4 Verification Limitations

TypeScript + linting are necessary but insufficient. Passing static checks ≠ runtime correctness. Most projects lack adequate test coverage.

Key Insight

We tested GPT-4o-mini and GPT-4o directly on real Supabase components with neutral refactoring prompts.

Both models dropped the `<Form_Shadcn_>` context provider — a hallucination invisible to TypeScript that silently breaks all form validation at runtime.

The answer to *"can an LLM refactor React code reliably with a generic prompt?"* is **no**.

Conclusion: Single-shot LLM refactoring is unsafe. We need structured coordination: classification, planning constraints, verification gates, and self-correction.

FOUNDATION

Related Works

STATE OF THE ART

Recent work spans **AST-based React smell detection** (ReactSniffer, IST 2023), **TypeScript/TSX extensions** (IST 2025), **AST vs AI-based correction** comparisons (SSRN 2025), and **MANTRA-style multi-agent refactoring** systems for end-to-end automated repair.

Contextual Gap

While detection is mature, reliable **automated remediation** at scale remains challenging due to framework-specific structural invariants that general-purpose LLMs often violate.

The Research Frontier

Current tools either rely on rigid AST transforms (low flexibility) or raw LLM prompts (low reliability). ReactRefactor bridge this by using **Agentic Workflows** to apply AST-validated framework maneuvers.

Focus: Moving from "Detecting technical debt" to "Reliable, autonomous debt retirement" in complex frontend ecosystems.

INQUIRY

Research Questions — Phase I

RQ1 **Actionability of Detected Smells**
To what extent are the smells detected by static analysis (ReactSniffer) genuinely refactorable in a production context?

RQ2 **Success Rate (Pass@k)**
How effectively does the multi-stage agentic pipeline generate correct, bug-free refactoring solutions on first and subsequent attempts?

RQ3 **Safety & Hallucination Avoidance**
Does the pipeline maintain structural integrity and avoid common LLM pitfalls like dropping context providers or inventing non-existent hooks?

INQUIRY

Research Questions — Phase II

RQ4 **Computational Cost & Scalability**
What is the economic viability of agentic refactoring compared to manual developer time? Does it scale linearly with project size?

RQ5 **Pipeline vs. Naive LLM Baseline**
Does adding agentic structure, planning, and verification provide a measurable safety gain over standard single-prompt LLM interactions?

←

→

ARCHITECTURE

System Design — Layers 1 & 2

LAYER 1 VS Code Extension

TypeScript

CORE MODULES

extension.ts
`src/extension.ts`

Main entry point. Registers all commands, wires event handlers, and orchestrates the scan, fix, and diff flow. Manages context keys that control which toolbar buttons are visible.

`activate()` `deactivate()` `SSE reader`

SmellsProvider
`src/smellsProvider.ts`

Implements the TreeDataProvider interface. Owns the full smell list, checkbox selections, fix statuses per smell, active filter, critique scores, and rejection reasons.

`TreeDataProvider` `setFilter()` `selectTopN()`

ServerManager
`src/serverManager.ts`

Detects a valid Python 3.8 or higher runtime, kills any stale process on port 7432, spawns the FastAPI server as a child process, and health-polls every 250ms until ready.

`detectPython()` `startServer()` `stopServer()`

VS CODE CONTRIBUTIONS (PACKAGE.JSON)

Activity Bar

Sidebar panel with Code Smells tree view. Auto-activates on workspaces containing .tsx, .jsx, or .js files.

Commands

Scan, Fix Selected, Cancel Fix, Select Top 100, Filter by Type, Revert Fix, Open Smell, View Diff.

Settings

openaiApiKey stored via VS Code secret storage. Falls back to OPENAI_API_KEY environment variable.

LAYER 2 FastAPI Server

Python, server/app.py, Port 7432

API ENDPOINTS

METHOD	ENDPOINT	DESCRIPTION	CALLED FROM
GET	/health	Returns server status and version. Polled every 250ms on startup.	serverManager.ts
POST	/scan	Runs detect_smells.py, enriches line ranges, applies severity rules, returns smell list as JSON.	runScan()
POST	/fix	Accepts smell tasks, creates a job, launches the LangGraph pipeline. Returns a job ID.	fixSelected cmd
GET	/progress/{job_id}	Server-Sent Events stream. Emits per-smell status changes as the pipeline progresses.	SSE reader
POST	/cancel/{job_id}	Sets a cancellation flag. In-flight pipeline runners check this flag and abort early.	cancelFix cmd
POST	/revert	Restores the file from a backup created before the fix was applied.	revertSmell cmd
GET	/original	Runs git show HEAD on the file for the diff view. Returns 404 if workspace is not a git repo.	viewDiff cmd

ARCHITECTURE

System Design — Layers 3 & 4

LAYER 3 Smell Detection

agentic_refactor_system/scripts/

detect_smells.py

`scripts/detect_smells.py`

Orchestrates the full detection flow. Invokes ReactSniffer CLI, reads the two CSV output files, deduplicates results, and applies path filters and severity rules before returning smells to the server.

ReactSniffer CLI

`vendor/reactsniffer/`

AST-based Node.js CLI bundled inside the VSIX package. Statically analyzes React components and outputs two CSV files: components_smells.csv and files_smells.csv.

AST-based · CSV output · Bundled in VSIX

Heuristic Fallback

`detect_smells.py`

Activates automatically when ReactSniffer produces no CSV output. Applies file-level heuristics to estimate smells. Results are less precise than AST analysis.

Auto-activates · File-level

Path Classifier

EXCLUDED ENTIRELY

- `node_modules/`
- `out/`
- `__generated__/`
- `.cache/`

DOWNGRADED TO LOW SEVERITY

- `packages/*/`
- `design-system/`
- `ui-kit/`
- `ui-components/`
- `component-library/`
- `primitives/`

LAYER 4 LangGraph Fix Pipeline

agentic_refactor_system/langgraph_pipeline/

PIPELINE NODE GRAPH

Classify `classify.py` — Decides if the smell is fixable or should be skipped

→ **Plan** `plan.py` — Generates a step-by-step fix strategy using GPT-4o

→ **Edit** `edit.py` — Applies code changes to the file using the plan

→ **Verify** `verify.py` — Reruns ReactSniffer to confirm the smell is resolved

→ **Critique** `critique.py` — LLM code review, assigns a quality score

→ **Finalize** `finalize.py` — Writes the file and emits an SSE status event

↩ If critique score is too low, routes back to Plan for another attempt

SUPPORTING MODULES

`state.py`
LangGraph state schema. Carries smell metadata, plan, edited code, verify result, critique score, and retry count between nodes.

`runner.py`
Async executor that runs the graph per smell, emits progress events over SSE, and checks the cancellation flag.

`tactics.py`
Maps each smell type to a recommended fix strategy, which is fed into the Plan node as additional context.

ARCHITECTURE

System Design — Layer 5

LAYER 5 External Dependencies

Runtime requirements

OpenAI API

GPT-4o powers the Classify, Plan, Edit, and Critique pipeline nodes. The API key is read from VS Code settings or the OPENAI_API_KEY environment variable.

GPT-4o
Chat Completions

Git CLI

Used by the /original endpoint to retrieve pre-fix file content via git show HEAD. Returns a 404 response gracefully when the workspace is not a git repository.

git show
subprocess

Node.js

Required to run the bundled ReactSniffer CLI. Invoked via npx subprocess from detect_smells.py. The ReactSniffer package is included in the VSIX bundle.

Node.js 16+
npx

Python 3.8+

Host machine runtime for the FastAPI server and the entire LangGraph pipeline. Detected automatically via the ms-python VS Code extension, then by searching PATH.

FastAPI
LangGraph
uvicorn

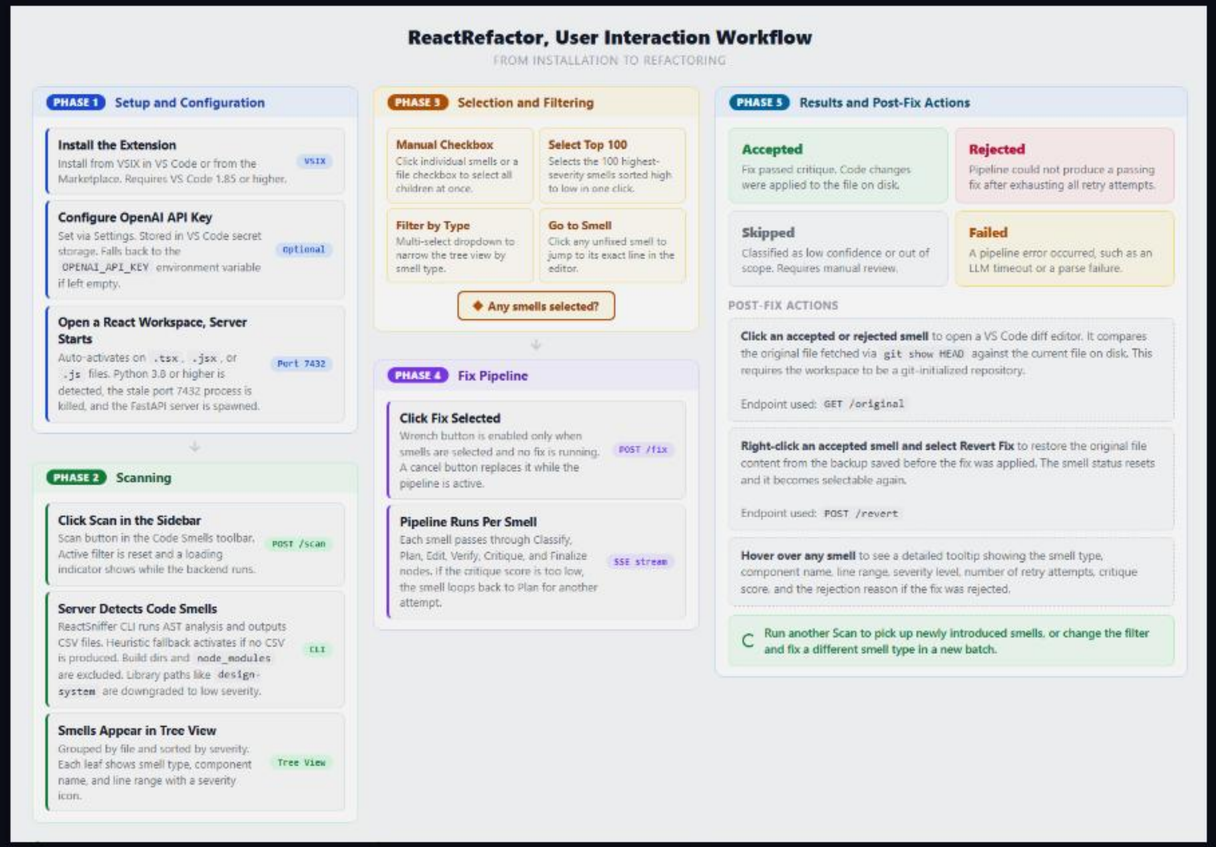
File System

The server reads and writes source files directly. A backup of the original file content is saved before any fix is applied, enabling the /revert endpoint to restore prior state.

read and write
backups

WORKFLOW

User Interaction Architecture



EVALUATION

Empirical Evaluation Methodology

EMPIRICAL METHODOLOGY

EXPERIMENT 1

Do React Refactors Preserve UI Behaviour?

1. **Synthetic React App** — Custom project built with ~20 deliberate React code smells across 3 routes and 8 distinct UI states
2. **Capture Baseline** — Before refactoring, capture screenshots, full DOM snapshots, and truth documents recording the position and presence of every UI component per state
3. **Apply Refactors** — Strong LLM refactors each smell using carefully constructed prompts with full file context. All code smells are resolved by the pipeline
4. **Capture Post-Refactor** — Same 8 states re-captured: screenshots, DOM snapshots, and truth documents after all refactors are applied
5. **Compare** — Diff screenshots pixel-by-pixel, compare DOM structure, and validate truth document component positions before vs after

3 Routes
Home, Customers, Reports

8 UI States
default, filtered, selected, help open, opened report, pinned ready

3 Artefact Types
PNG screenshots, HTML DOM snapshots, JSON truth documents

Goal
Confirm structural refactors produce zero observable UI change

EXPERIMENT 2

Agentic Pipeline vs. Weak LLM vs. Strong LLM (20 Smells)

CORPUS
20 smells, stratified random sample from 1,210 actionable smells in the Supabase codebase (seed = 42)

LLM CONFIG
`temperature=0` across all systems for reproducibility. Pipeline uses GPT-4o-mini for classify, plan, critique and GPT-4o for edit

PIPELINE GATES
`tsc --noEmit`, ReactSniffer re-run, `pnpm build`, critique score above 0.7, up to 3 retries

SYSTEM	MODEL	APPROACH	EVALUATION METHOD
Weak LLM	GPT-3.5-turbo	Direct neutral prompt. No tactic guidance, no verification step, single shot	LLM-as-judge
Strong LLM	GPT-4o	Direct neutral prompt. No tactic guidance, no verification step, single shot	LLM-as-judge
Agentic Pipeline	GPT-4o-mini + GPT-4o	Classify, Plan (tactic-constrained), Edit (bounded scope), Verify, Critique with retry loop	Automated gates

EXPERIMENT 3

End-to-End VS Code Extension Runs on Real-World Repositories

Tool
ReactRefactor VS Code extension, end-to-end from scan to accepted fix

Repo Selection
Three open-source React projects chosen to represent small, medium, and large codebase scales

Procedure
Scan each repo, select smells using the extension, run the full agentic pipeline, record accepted, rejected, skipped, and failed outcomes

SMALL
JUMBLE
github.com/Shikha307/JUMBLE
23 identified code smells
Small hobby-scale React project. Used to validate the pipeline on a minimal codebase with limited component complexity.

MEDIUM
Excalidraw
github.com/excalidraw/excalidraw
101 identified code smells
Widely-used open-source diagramming tool. Represents a mid-scale production React codebase with diverse component patterns.

LARGE
Supabase
github.com/supabase/supabase
1,137 identified code smells
Large-scale production monorepo with modern React patterns including shadcn/ui, react-hook-form, and Radix UI primitives.

RESULTS

Quantitative Evaluation Summary

EXPERIMENT 1: PERFORMANCE ACROSS REPOSITORIES

REPOSITORY	ACCEPTED	REJECTED	SKIPPED/FAILED	PASS RATE
Jumble (Selection)	16	7	0	**69.6%**
Excalidraw	95	1	4	**95.0%**
Supabase	99	0	1	**99.1%**
TOTAL	**210**	**8**	**5**	**94.1%**

EXPERIMENT 2: BASELINE COMPARISON (PASS@1)

METRIC	WEAK LLM (GPT-4O-MINI)	STRONG LLM (GPT-4O)	OUR PIPELINE
Overall Pass Rate	7/20 (35%)	6/20 (30%)	**14/20 (70%)**

Key Takeaways

WHAT WORKS

Isolated Smells: Near-perfect acceptance for standalone components (Navbar/Dashboard).
Direct Tactics: `add_controlled_state` achieves 100% success with zero retries.

THE CHALLENGE

Entanglement: Multi-component files (Matches.jsx) cause surgical split conflicts.
Dirty State: Pre-existing uncommitted changes break patch verification gates.

SURPRISING INSIGHTS

Failures: Often path hallucinations (truncated directory names) rather than logic complexity.
Latency: 10x deviation in large files (AuditLogs.tsx) due to context pressure.

OUTLIERS & TRIAGE

Implicit Triage: 4 smell categories (Force Update, DOM Manip) were systematically untouched.
Invalid Plans: Plan-level errors remain unrecoverable by the standard retry loop.

LESSONS LEARNED

Validation: Path normalization must precede scope validation.
Strategy: Reliability is high for *self-selected* subsets; full-scale coordination is the next frontier.

CONCLUSION

Scalability proven for isolated tasks ($0.005/refactor). **Entangled files** and **global coordination** are the primary failure frontiers.

DEMO

←